CS 300 – Chapter 12 Notes

Sets

Jose de Lima

April 21, 2026

Key idea

The set ADT in one sentence. A set is a collection of distinct elements with fast membership queries. It's the data structure you reach for any time you care about "have I seen this before" or "how do these two collections compare," and almost nothing else. Everything in this chapter is about the operations that make sense on top of that single primitive.

1 12.1 The Set ADT

What a set is

Definition —

Set. An unordered collection of distinct elements. No duplicates, no position. $\{3, 7, 9\} = \{9, 7, 3\}$.

The three things a set gives you:

- `add(x)`: insert x if not already present; no-op otherwise.
- `remove(x)`: remove x if present.
- `contains(x)`: membership query.

That's it. Everything else (union, intersection, filter, map, subset) is built on top of these three.

Keys vs. elements

When the elements are primitives (ints, strings), each element is its own key. When the elements are objects – a `Student` with `name`, `phone`, `id` – the set uses one field as the key and compares elements by that. The rest of the object is just cargo.

```
// C++ lets you choose: the element type IS the key, or you supply
// a hash/comparator that extracts a key.
std::unordered_set<int> primes;
std::unordered_set<Student, StudentHash, StudentEq> roster;

// Or use a map if you need key/value separation.
std::unordered_map<int, Student> rosterById;
```

Aside

In Python, sets use `__hash__` and `__eq__`; in Java, it's `hashCode()` and `equals()`. The contract is always the same: equal elements must hash equal. If you break that contract, the set is not a set – it's a corrupt data structure waiting to lose an element.

Subset testing

$X \subseteq Y$ iff every element of X is in Y. Trivial algorithm:

```
bool isSubset(const std::unordered_set<int>& sub,
              const std::unordered_set<int>& sup) {
    for (int x : sub)
        if (!sup.count(x)) return false;
    return true;
}
```

Cost: $O(|X|)$ if `contains` is $O(1)$ (hash set). If `sup` is a sorted structure (tree set), it's $O(|X| \log |Y|)$.

2 12.2 Set Operations

Union, intersection, difference

Definition —

$X \cup Y$: everything in either set. $X \cap Y$: everything in both sets. $X \setminus Y$: everything in X but not in Y.

Union and intersection are commutative ($X \cup Y = Y \cup X$). Difference is not.

Example

$\{54, 19, 75\} \cup \{75, 12\} = \{12, 19, 54, 75\}$
$\{54, 19, 75\} \cap \{75, 12\} = \{75\}$
$\{54, 19, 75\} \setminus \{75, 12\} = \{19, 54\}$
$\{75, 12\} \setminus \{54, 19, 75\} = \{12\}$

C++ implementations

The `<algorithm>` header provides set operations on *sorted ranges*, which means any sorted container:

```
#include <set>
#include <algorithm>
#include <iterator>

std::set<int> a{54, 19, 75}, b{75, 12}, out;

std::set_union(a.begin(), a.end(), b.begin(), b.end(),
               std::inserter(out, out.end()));
// out = {12, 19, 54, 75}

out.clear();
std::set_intersection(a.begin(), a.end(), b.begin(), b.end(),
```

```
                        std::inserter(out, out.end()));
// out = {75}

out.clear();
std::set_difference(a.begin(), a.end(), b.begin(), b.end(),
                    std::inserter(out, out.end()));
// out = {19, 54}
```

For `std::unordered_set`, you write the loops explicitly:

```
std::unordered_set<int> setUnion(const std::unordered_set<int>& a,
                                 const std::unordered_set<int>& b) {
    std::unordered_set<int> r(a);
    r.insert(b.begin(), b.end());
    return r;
}

std::unordered_set<int> setIntersect(const std::unordered_set<int>& a,
                                     const std::unordered_set<int>& b) {
    // Iterate the smaller one -- saves time.
    const auto& smaller = a.size() <= b.size() ? a : b;
    const auto& larger  = a.size() <= b.size() ? b : a;
    std::unordered_set<int> r;
    for (int x : smaller)
        if (larger.count(x)) r.insert(x);
    return r;
}

std::unordered_set<int> setDiff(const std::unordered_set<int>& a,
                                const std::unordered_set<int>& b) {
    std::unordered_set<int> r;
    for (int x : a)
        if (!b.count(x)) r.insert(x);
    return r;
}
```

Gotcha

Always iterate the smaller set. For intersection, this shaves the cost from $O(|A|)$ to $O(\min(|A|, |B|))$ – a huge win when one side is tiny. It's the single most common optimization in real set-heavy code.

Cost analysis

Assume hash-based sets with $O(1)$ expected insert and lookup.

Operation	Cost	Why
Union $\lvert A\rvert + \lvert B\rvert$	$O(\lvert A\rvert + \lvert B\rvert)$	insert all
Intersection	$O(\min(\lvert A\rvert, \lvert B\rvert))$	scan smaller, probe larger
Difference $A \setminus B$	$O(\lvert A\rvert)$	scan A, probe B
Subset $A \subseteq B$	$O(\lvert A\rvert)$	scan A, probe B

For sorted (tree) sets, add a $\log n$ factor for each membership probe.

Key idea

Sorted vs. hashed, revisited. With sorted sets, the two-pointer merge algorithm computes union/intersection/difference in $O(|A| + |B|)$, matching the hashed version without any per-element hash. On small sets that fit in cache, the sorted version can actually be *faster*, because hashing has nontrivial constant overhead. For large sets with large element types, hashed wins. Benchmark before picking.

Filter and map

Two higher-order operations borrowed from functional programming:

Definition —

Filter. Given a predicate p, return the subset of elements for which p is true.
Map. Given a function f, return the set $\{f(x) : x \in X\}$. The result may be smaller than the input if f is not injective (multiple inputs can map to the same output).

```
#include <unordered_set>
#include <functional>

template<typename T, typename Pred>
std::unordered_set<T> setFilter(const std::unordered_set<T>& s, Pred p) {
    std::unordered_set<T> r;
    for (const auto& x : s) if (p(x)) r.insert(x);
    return r;
}

template<typename T, typename U, typename Fn>
std::unordered_set<U> setMap(const std::unordered_set<T>& s, Fn f) {
    std::unordered_set<U> r;
    for (const auto& x : s) r.insert(f(x));
    return r;
}

// Usage:
auto evens = setFilter(nums, [](int x){ return x % 2 == 0; });
auto lengths = setMap<std::string, size_t>(words,
                    [](const std::string& w){ return w.size(); });
```

Python / JavaScript / Rust / Haskell / Scala all have these as first-class methods on their set (and collection) types. C++ got closer in C++20 with `std::ranges::filter_view` and `std::ranges::transform_view`, but the syntax is still clunky compared to `xs.filter(...).map(...)` in better-styled languages.

Aside

Filter is a subset; map is not always. Filter always returns a subset of the original. Map can shrink the set if collisions occur under $f\colon \{1, -1\} \to \{1\}$ via $f(x) = x^2$. If you need one-to-one semantics, use a list or multiset and track duplicates explicitly.

3 12.3 Static vs. Dynamic Sets

Definition —

Dynamic set. Can be modified after construction – supports add and remove. **Static set.** Fixed at construction time. Supports read-only and new-set-producing operations (union, intersection, filter, …), but not in-place add or remove.

Operation	Dynamic	Static
Construct from collection	yes	yes
Count elements	yes	yes
Search / contains	yes	yes
Add	yes	no
Remove	yes	no
Union / intersection / difference (return new)	yes	yes
Filter / map (return new)	yes	yes

Why would you ever want a static set?

Key idea

A static set is *immutable*. That buys you several things a mutable set can't:

- **Thread safety for free.** If nothing can change, no locks are needed.
- **Aggressive optimization.** The implementation can use a perfect hash (no collisions), a sorted array (cache-friendly, no per-bucket pointers), or compile-time constants.
- **Safe sharing.** Hand the same set to a dozen consumers and no one can break it.
- **Hashability.** An immutable set can itself be a key in another set or map. (Python's `frozenset` exists for exactly this.)

Choosing in practice

Use case	Choice
Allowed-country codes loaded at startup	Static – set once, read forever
Stop-words for a search engine	Static – fixed per language
Users currently online	Dynamic – changes constantly
Session IDs issued today	Dynamic – grows over time
Finalized vote tallies (after the election)	Static – sealed after a deadline
Shopping cart contents	Dynamic – user modifies it

C++ idioms for static sets

C++ doesn't have a built-in frozen set. Common workarounds:

```
// 1. Wrap in const.
const std::unordered_set<std::string> STOP_WORDS{"a", "an", "the", "of"};

// 2. C++17 initializer at static scope.
constexpr std::array<int, 4> ALLOWED_CODES{200, 301, 302, 404};
// Binary-search in a sorted array -- cache-friendly, no dynamic allocation.
```

```
// 3. Compile-time perfect hash via gperf, for pathological performance.
```

Aside

Immutability is a spectrum. "Const" in C++ is syntactic – someone can still `const_cast` it away. Python's `frozenset` is structurally immutable. Rust and Haskell make the distinction first-class. If your codebase has a convention that "const means truly immutable," lean into static sets; if not, you're one `const_cast` from heisenbugs.

Sets you'll actually use

Container	Order	Typical implementation
`std::set`	sorted	red-black tree (ch. 9)
`std::unordered_set`	hash order	open-addressed hash table
`std::multiset`	sorted, allows dups	red-black tree
Python `set`	hash order	open-addressed hash table
Python `frozenset`	hash order	immutable hash table
Java `HashSet` / `TreeSet`	–	hash / red-black
Rust `HashSet` / `BTreeSet`	–	hash / B-tree

Key idea

Chapter takeaway. Sets are the deduplicating, order-free, membership-query-oriented sibling of lists. Once you can think of a problem as "this is a collection where I only care about distinct membership," half the algorithm writes itself: iterate one side, probe the other. Pick hash-based when order doesn't matter and you want $O(1)$ membership; pick tree-based when you want sorted iteration or `lower_bound`; pick static/frozen when no one should be adding or removing elements. The ADT is simple; the engineering choices it unlocks are not.